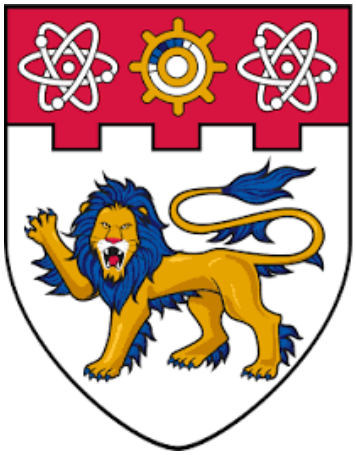


NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Layout-to-Image Generation with Multiple Objects: Comparing GLIGEN and MIGC on COCO-MIG

CA6127 Transformer and Generative Adversarial Networks
Group Project (Technical Review)

Group 7:

Zhichen Liu
Xinkuo Wu
Yanyu Wen

1. Introduction

Text-to-image diffusion models such as Stable Diffusion follow a prompt well, but they handle two aspects of multi-object scenes poorly: placing each object at a requested position, and assigning each object its own attribute. For a prompt such as "a red donut and a green donut and a blue donut", the model often produces donuts with the colours swapped, or fewer donuts than requested.

Layout-to-image (L2I) generation addresses this by adding a bounding box and a short phrase ("a red donut") for every object as extra conditioning. Because the boxes and phrases are known, the output can be checked automatically: whether a donut appears in the box, and whether it is red. With several objects per image the task is referred to as multi-instance generation (MIG).

This project compares two representative L2I methods that share the same backbone (Stable Diffusion 1.4) but inject the layout in different ways: GLIGEN (CVPR 2023) and MIGC (CVPR 2024). Because the backbone is identical, differences between the two are more likely to come from the control mechanism than from the generator. The project addresses three questions:

- Which method follows the layout better, and where exactly does the other one fail (missing object, wrong place, or wrong colour)?
- Which conditions make the task hard: many objects, overlapping boxes, same category in different colours, small objects?
- Can the weaker method be improved by a cheap fine-tuning of its layout-related modules, and which hyper-parameters matter?

All images, evaluations and analyses reported here were produced by the group with the code in the submitted package. Results that could not be independently re-verified are marked as such in the text.

2. Methods

Both methods keep the SD 1.4 UNet frozen and add trainable modules. Tensor shapes were recorded with forward hooks on a 2-object example, so the descriptions below follow the released code rather than the papers alone. Figure 1 shows where each method plugs into the network.

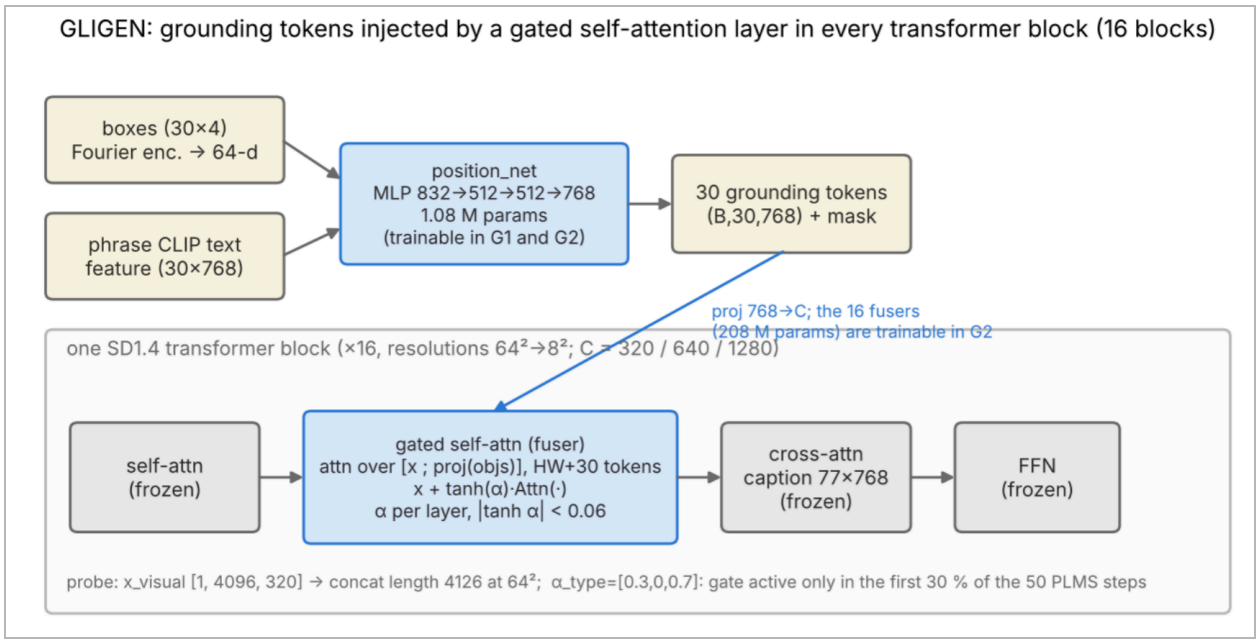


***Figure 1a**. GLIGEN: a "gated self-attention" layer is inserted in every one of the 16 transformer blocks. Box + phrase are turned into 30 grounding tokens by position_net.*

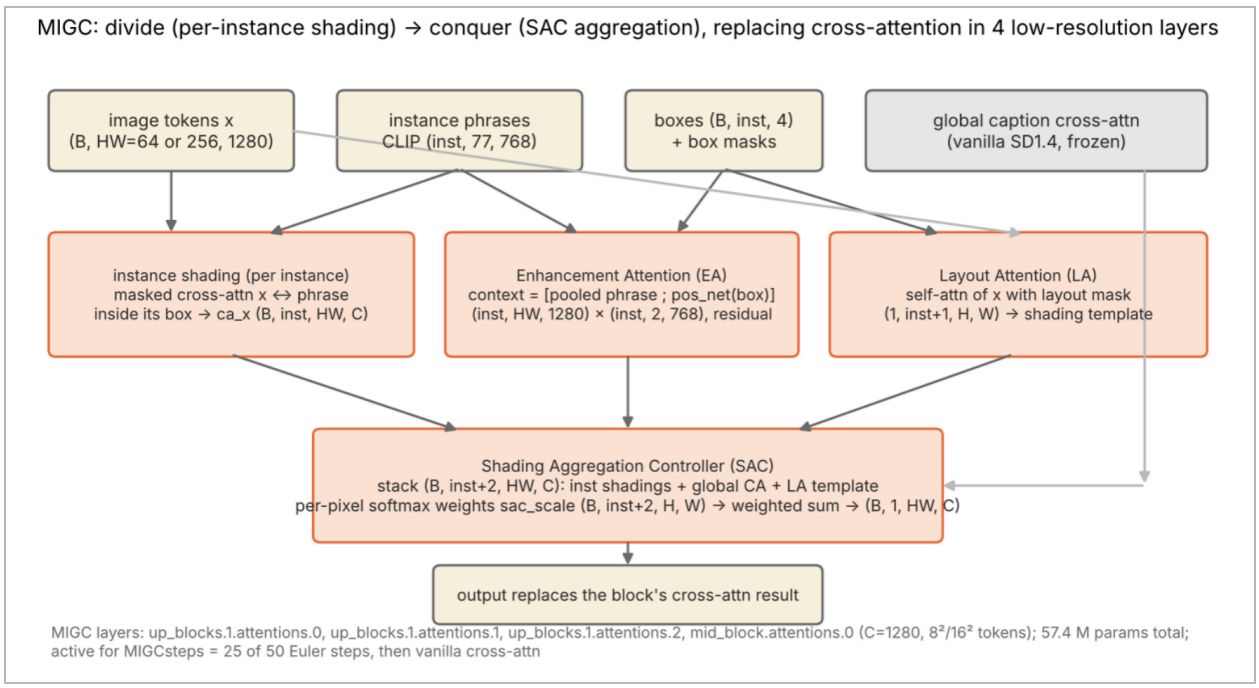


***Figure 1b**. MIGC: cross-attention in 4 low-resolution layers is replaced by a per-instance controller (instance shading, Enhancement Attention, Layout Attention) whose outputs are blended by the Shading Aggregation Controller.*

2.1 GLIGEN

GLIGEN encodes each (box, phrase) pair into one "grounding token": the four box coordinates are Fourier-encoded (64 numbers), concatenated with the CLIP text feature of the phrase (768 numbers), and passed through a small MLP (832→512→512→768). There are always 30 token slots; unused slots are filled with a learned "null" token. This module, position_net, has only 1.08 M parameters.

Inside every transformer block, after the normal self-attention, the image tokens are concatenated with the 30 grounding tokens and go through one more self-attention layer (the "fuser"). Its output is added back through a learned scalar gate: $x \leftarrow x + \tanh(\alpha) \cdot \mathrm{Attn}([x; \text{tokens}])$. The 16 fusers hold 208 M parameters (19.5% of the UNet). In the official checkpoint all gate values $|\tanh \alpha|$ are below 0.06. At inference the gate is only open for the first 30 % of the sampling steps (15 of 50); after that the model behaves like plain SD 1.4.

Note that nothing in this design ties the colour word of a phrase to its box: the phrase is encoded as a whole, and the global caption, which contains all colour words, is still processed by the frozen cross-attention.

2.2 MIGC

MIGC only touches 4 layers: the mid block (8×8 tokens) and the three attention layers of up_blocks.1 (16×16 tokens), all with 1280 channels. In total 57.4 M parameters are added (6.7% of the UNet). Inside these layers MIGC follows a divide-and-conquer scheme:

- **Instance shading.** For every instance a separate masked cross-attention is computed between the image tokens and that instance's phrase, restricted to its box. Thus the phrase "a red donut" only interacts with the pixels inside its own box.
- **Enhancement Attention (EA).** Each instance attends once more to a 2-token context (the pooled phrase and a box token), which strengthens the response inside the box.
- **Layout Attention (LA).** A self-attention over the image tokens with a layout mask, producing a shared "shading template" (consistent background/lighting).
- **Shading Aggregation Controller (SAC).** All candidates (one per instance + the normal caption attention + the LA template) are stacked and blended with learned per-pixel softmax weights. MIGC is active for the first 25 of 50 steps.

In summary, GLIGEN lets all grounding tokens interact inside one attention layer and relies on the frozen caption attention for appearance, whereas MIGC keeps the instances separate until a learned blending step merges them.

	GLIGEN	MIGC
How the layout enters	30 tokens concatenated into self-attention	one masked cross-attention per instance
Layers touched	all 16 blocks (64^2 down to 8^2)	4 layers (16^2 and 8^2)
Spatial constraint	implicit (box is just part of a token)	explicit (box mask + SAC blending)
Instances isolated?	no	yes, until SAC

Added parameters	209 M (19.6%)	57.4 M (6.7%)
Control active for	15/50 steps (30 %)	25/50 steps (50 %)
Official sampler	PLMS	Euler
Speed / memory (RTX 4070)	5.7 s per image, 9.4 GB	6.5 s per image, 7.8 GB

Table 1. *Implementation facts (from the official code and our probes).*

3. Experimental setup

3.1 Dataset

We use COCO-MIG, the benchmark introduced with MIGC. It contains 800 layouts taken from COCO val2014 boxes; each object is assigned one of seven colours (red, blue, green, yellow, black, white, brown) and images have 2 to 6 objects (levels L2–L6). The layouts were split by level into a test set of 100 layouts (20 per level), a validation set of 30 layouts used only for hyper-parameter choices, and a 10-layout pilot subset for debugging. The splits are fixed in the repository (with sha256 hashes). Every layout is generated with two random seeds, so each method produced 200 test images containing 800 target objects.

From the inputs alone, each object was tagged with difficulty conditions: whether the same category appears in a different colour in the same image (57 % of objects), how much its box overlaps with another box (none / low / mid / high), and its size quartile. COCO-MIG contains no "small" objects in the COCO sense (the smallest box is 66 px wide at 512×512), so size is analysed by quartile.

For fine-tuning, 3,000 images were taken from COCO train2014. train2014 and val2014 are disjoint image sets; in addition, every evaluation layout was searched against the training images for near-duplicates (0 of 130 matched at mean IoU $\geq$ 0.9). COCO image IDs could not be checked one by one because the benchmark file does not record them.

3.2 Generation settings

All settings shared by the two methods were fixed: 512×512, 50 steps, CFG 7.5, the same negative prompt, the same initial noise for a given seed, fp16 autocast. Both wrappers reproduce the official inference scripts pixel-for-pixel. Two settings were left at each method's official value and therefore differ: the sampler (GLIGEN's code uses PLMS, MIGC's uses Euler) and how long the layout control is active (GLIGEN 15/50 steps, MIGC 25/50). Both differences are discussed in the limitations.

3.3 Evaluation

The official MIG-Bench protocol was re-implemented: Grounding DINO detects objects of the requested category; an object is "in position" if some detection overlaps the target box with IoU $\geq$ 0.5; SAM segments the object inside the target box; the colour is "correct" if at least 20 % of the mask pixels fall in the HSV range of the requested colour. The main metric, referred to here as Attribute Success (identical to MIG-Bench instance success), requires both position and colour to be correct. Image Success means every object in the image succeeded.

A few diagnostics were added under separate names so that they are not confused with the official metrics: Object Recall (the object was drawn somewhere — a one-to-one match exists), Localisation IoU (position quality ignoring colour), the failure type of each object (missing / mislocated / colour wrong),

and Color Leakage Rate (the object's mask is dominated by a colour that was requested for another object in the same image). Image quality is measured with CLIP-IQA.

All confidence intervals are 95 % bootstrap intervals computed over layouts (both seeds and all objects of a layout are resampled together, 5,000 draws). Differences between methods are paired on the same (layout, seed, object).

4. Results and analysis

4.1 Overall comparison

Table 2 and Figure 2 give the main result. [1]

Metric	GLIGEN	MIGC	MIGC − GLIGEN [95 % CI]
Object Recall ↑	0.917	0.932	+0.015 [−0.012, +0.042]
Position Success ↑	0.809	0.873	+0.064 [+0.027, +0.102]
Localisation IoU ↑	0.690	0.741	+0.051 [+0.023, +0.080]
Attribute Success ↑	0.321	0.665	+0.344 [+0.294, +0.393]
given position OK	0.397	0.762	+0.365 (unpaired)
Official mIoU ↑	0.288	0.572	+0.284 [+0.243, +0.325]
Image Success ↑	0.065	0.280	+0.215 [+0.150, +0.280]
Color Leakage Rate ↓	0.279	0.091	−0.188 [−0.229, −0.142]
Missing Rate ↓	0.033	0.009	−0.024 [−0.042, −0.006]
Mislocation Rate ↓	0.159	0.119	−0.040 [−0.078, −0.005]
CLIP-IQA ↑	0.689	0.707	+0.018 [+0.002, +0.034]

Table 2. Test set (100 layouts × 2 seeds, 800 objects per method). Last column: paired difference with layout-bootstrap CI.

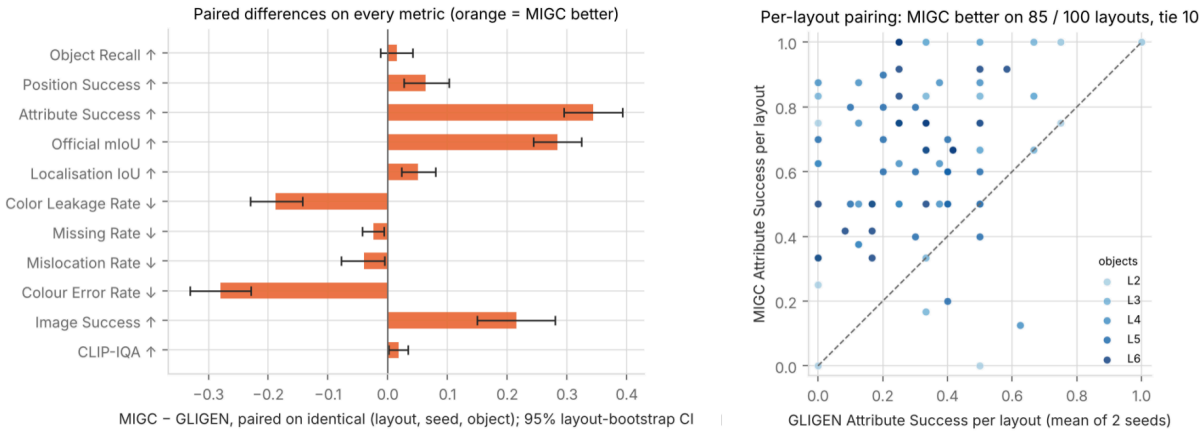


Figure 2. *Left: paired difference MIGC − GLIGEN on every metric (orange = MIGC better). Right: per-layout Attribute Success; MIGC is better on 85 of 100 layouts.*

Three observations follow from these results:

Both methods draw the objects. Object Recall is 0.92 vs 0.93, and the difference is not significant. MIGC places them slightly more accurately (Position Success +0.06, IoU +0.05).

The gap is in colour. Among objects that are in the right place, only 40% of GLIGEN's have the right colour versus 76% of MIGC's. The 0.34 gap in Attribute Success is therefore accompanied mainly by a difference in colour pass rate, while position differences are small. This is not a causal decomposition: colour is only judged after the position test has been passed.

Image-level success amplifies instance-level differences. With about 4 objects per image, GLIGEN produces a fully correct image in 0.065 of cases and MIGC in 0.280. Image quality is essentially unchanged (CLIP-IQA difference +0.018), so MIGC's stronger control does not come at the expense of fidelity.

4.2 Failure modes

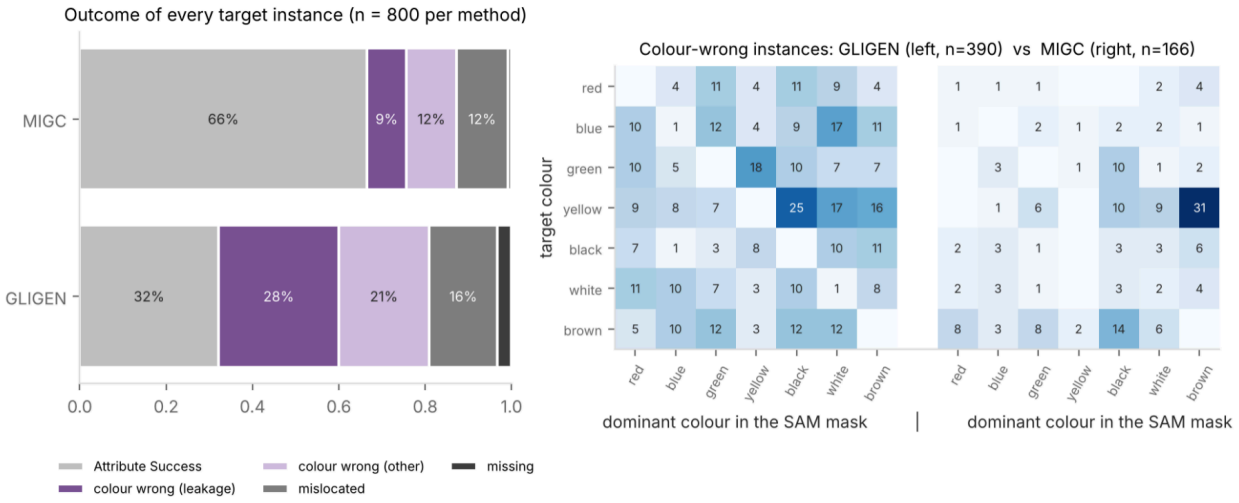


***Figure 3**. Left: what happened to every target object. Right: for colour-wrong objects, the requested colour (rows) vs the colour that actually dominated the mask (columns).*

GLIGEN: 49% of all objects are colour-wrong, and 57% of those are leakage — the object took a colour that was requested for a different object. The confusion matrix in Figure 3 (left) is almost uniform: any colour can end up on any object. This is consistent with the architecture: the grounding tokens share one attention layer, and the colours are ultimately rendered by the frozen caption attention, which sees "a red donut and a green donut and …" with nothing tying "red" to a particular box. This is a plausible explanation rather than a result established by ablation.

MIGC: 21% colour-wrong, 9% leakage, and 33% of its colour errors (55 of 166) sit on just three pairs of neighbouring HSV colours: yellow→brown (31), brown→black (14) and yellow→black (10). Visual inspection (Figure 4, right) shows tan-coloured cows judged "brown" and clearly brown vases judged "red" by the evaluator. Part of MIGC's remaining error is therefore a limitation of the HSV colour table rather than of the model.

Missing and mislocated objects are rare for both methods (GLIGEN 3% / 16%, MIGC 1% / 12%). Several GLIGEN mislocations are objects that extend beyond their box; this may be related to the 70 % of steps during which its gate is closed, although this was not tested.



Figure 4 *(left). Category A: GLIGEN colour leakage on 6-object layouts where MIGC gets everything right.*



Figure 4 *(right). Category E: MIGC objects judged colour-wrong on yellow/brown boundaries, where the evaluator's colour table is arguably at fault. Boxes show the target layout in the target colour; the label is the evaluator's verdict.*

4.3 Difficulty conditions

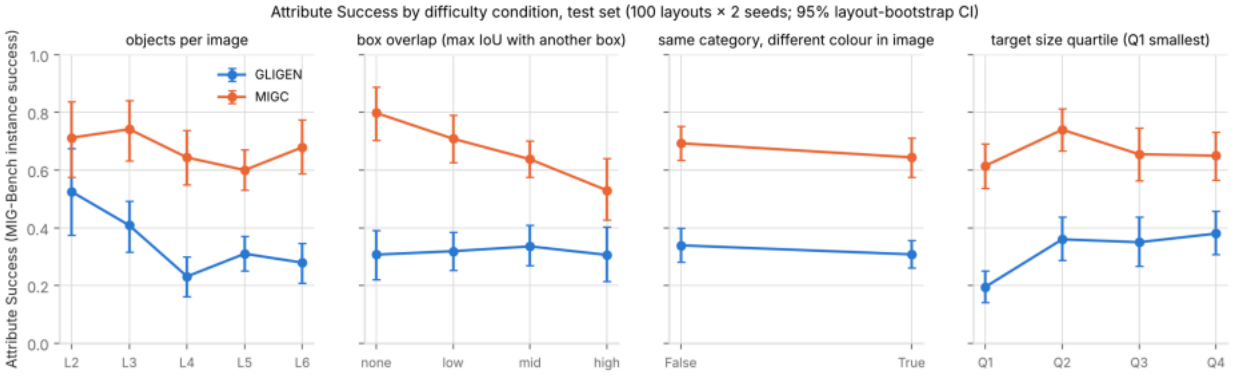


Figure 5. Attribute Success by difficulty condition (95 % layout-bootstrap CIs).

1) **Number of objects** is GLIGEN's main difficulty: 0.53 with 2 objects down to 0.28 with 6, while MIGC stays flat (0.71 → 0.68). A plausible reading: more objects means more colour words competing inside one shared attention, whereas MIGC's instance branches do not see each other.
2) **Box overlap** is MIGC's main difficulty: 0.80 with no overlap down to 0.53 with high overlap, so the gap to GLIGEN shrinks from +0.49 to +0.22 [+0.09, +0.35]. This contradicts our prior expectation that the SAC blending step would handle overlaps well. One possible explanation, not tested here, is that in an overlapping region two instance branches each paint their own colour and the weighted blend dilutes both, so one of them drops below the 20 % colour threshold. GLIGEN is flat here, possibly because it is already close to its floor.
3) **Same category, different colour** changes neither method by more than 0.05. Colour errors are not specifically a same-category problem.
4) **Small targets** (smallest quartile, box area below 5.1 % of the image) hurt GLIGEN substantially (0.20 vs 0.38 for the largest quartile) but not MIGC (0.61 vs 0.65), even though MIGC only controls at 8×8 / 16×16 resolution where such a box covers only about 4–13 of the 256 tokens. This suggests that how the instances are kept apart matters more than the resolution of the control.

4.4 Evaluator sensitivity

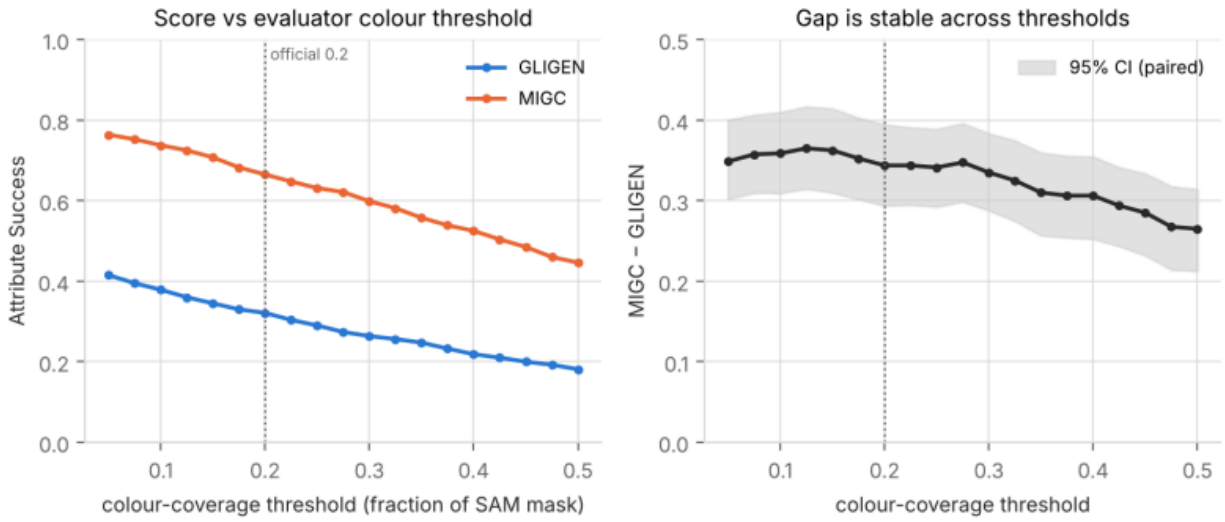


***Figure 6**. Changing the evaluator's colour threshold (images, detections and masks fixed). The scores move; the gap does not.*

The headline metric depends on a hand-picked threshold (20 % of mask pixels in the target colour range). Because the evaluator caches the colour ratio of every object, Attribute Success could be recomputed for thresholds from 0.05 to 0.5 without any GPU computation. Absolute scores fall almost linearly (GLIGEN 0.41 → 0.18, MIGC 0.76 → 0.45), but the paired gap stays between 0.27 and 0.36 and its intervals never include zero. The ranking is therefore robust, but absolute numbers must be reported together with the threshold. The cache cannot be used to change the position threshold or the detector; that would require new GPU evaluation.

4.5 Fine-tuning GLIGEN's layout modules

GLIGEN's weak point is colour binding, which raises the question of whether its two layout modules can be adapted cheaply. Starting from the official checkpoint, we fine-tuned (G1) only position_net (1.08 M parameters, 0.10% of the UNet) and (G2) position_net + the fusers (209 M, 19.6%) on 3,000 COCO train2014 images. The phrases only contain the category name (e.g. "dog"), so there is no colour supervision. Shared settings: 2,000 steps, effective batch 8, AdamW without weight decay, 100 warm-up steps then a constant learning rate, bf16, the official ε-MSE objective with 10 % grounding dropout, optimiser state reset; checkpoints at 500 and 2,000 steps; G2 was run with learning rates 5e-5 and 1e-5. All weight and gradient checks passed (frozen parameters unchanged, every trainable parameter updated).

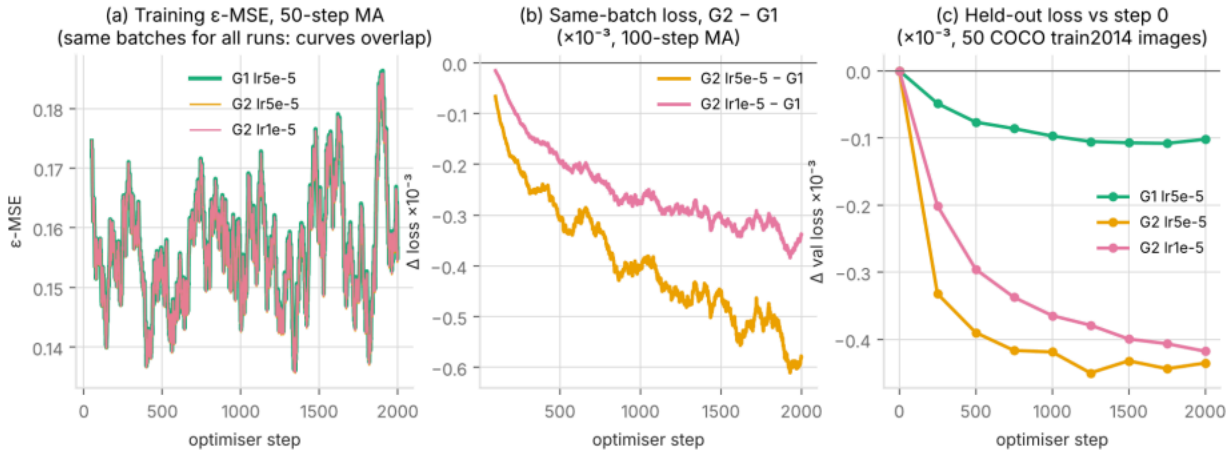


***Figure 7**. Training. (a) All runs see identical batches, so the loss curves overlap. (b) The same-batch difference shows G2 fits slightly better than G1. (c) Held-out loss on 50 COCO images saturates after about 1,250 steps. The absolute changes are below 0.3 % — expected with a frozen SD and no attribute supervision.*

Run	Attr. Success	Δ vs official [95 % CI]	Colour | position	Loc. IoU
official	0.279	--	0.310	0.746
G1 5e-5 @500	0.321	+0.042 [+0.004, +0.080]	0.363	0.747
G1 5e-5 @2000	0.304	+0.025 [−0.012, +0.064]	0.340	0.755
G2 1e-5 @500	0.287	+0.008 [−0.032, +0.047]	0.322	0.753
G2 1e-5 @2000	0.296	+0.017 [−0.028, +0.065]	0.327	0.764
G2 5e-5 @500	0.329	+0.050 [+0.009, +0.096]	0.369	0.751
G2 5e-5 @2000	0.325	+0.046 [−0.016, +0.111]	0.375	0.737

Table 3. Fine-tuned GLIGEN on the validation set (30 layouts × 2 seeds, grounding ratio 0.5). Exploratory — only 30 layouts.

Learning rate. With 1e-5, G2 is indistinguishable from the official model. With 5e-5 the validation point estimates are higher, but the direct contrast (G2 5e-5 vs 1e-5 at 500 steps: +0.042 [−0.004, +0.093]) includes zero. The data therefore only show that 1e-5 had no measurable effect, not that 5e-5 is better.

Checkpoint. No consistent advantage of 2,000 over 500 steps was found (G1 even drops from 0.321 to 0.304; all 2000 − 500 contrasts include zero). Held-out loss stops improving around 1,250 steps while the

weights keep drifting (position_net $\|\Delta\theta\|$ reaches 2.7). Intermediate checkpoints were not evaluated on the task, so the exact onset of the plateau is unknown.

Grounding ratio. The group's Stage C-1 log reports that extending GLIGEN's gate from 30 % to 50 % of the steps gave a small but significant gain in localisation IoU and a smaller, non-significant change in Attribute Success. The raw outputs of that experiment are not in the code package, so no numbers from it are quoted here and it is treated as unverified. All fine-tuned runs use the ratio 0.5 that was chosen from that log.

Where does the change appear? The largest point-estimate changes are in colour-given-position (0.31 → 0.34–0.38) and in leakage (G2 5e-5 @500: 0.36 → 0.30 [−0.10, −0.01]); localisation and object presence do not change consistently. Training only position_net gives about the same change as G2 (+0.042 vs +0.050) with 1/193 of the trainable parameters and 5.8 vs 8.4 GB of GPU memory. The gate values $\tanh(\alpha)$ barely move (< 0.005), which rules out a widening of the gate as the mechanism; whether the change sits in the fuser weights, in position_net, or in both cannot be determined from this evidence. With at most +0.05 on validation and no colour supervision, this kind of fine-tuning is unlikely to close a 0.34 gap. Our hypothesis, untested here, is that the binding problem lies in the frozen caption attention, which GLIGEN's design leaves untouched.

5. Discussion

5.1 Strengths and weaknesses of the two methods

GLIGEN is general (the same token mechanism can carry key-points or depth maps), it touches every resolution, and it has a clean training story (frozen SD, gates initialised at zero). However, its shared-attention injection has no notion of which instance owns which pixel: attributes leak, and performance drops with more objects and smaller boxes.

MIGC's per-instance branches give it strong attribute binding at low cost (6.7% extra parameters, 4 layers) and make it robust to the number of objects. However, its blending step degrades when boxes overlap, control only exists at 8×8 / 16×16 resolution, and because every instance gets its own branch, generation time grows with the number of objects (Figure 8), while GLIGEN's time is constant.

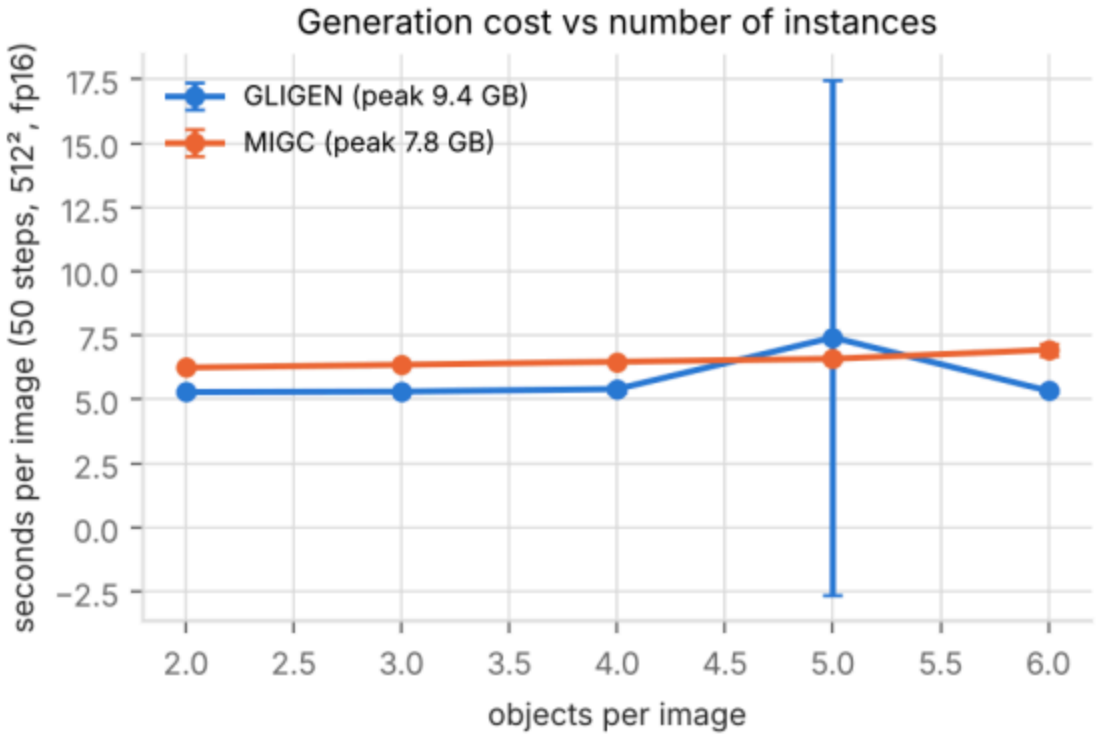


***Figure 8**. Generation time vs number of objects (RTX 4070, fp16, 50 steps).*

5.2 Limitations of our study

- No single-component ablation was run, so the architectural explanation of the gap is an interpretation consistent with the mechanism and the failure patterns, not an isolated causal result.
- 100 test layouts × 2 seeds; confidence intervals are not adjusted for multiple comparisons; the validation set has only 30 layouts, so Section 4.5 is exploratory and several of its contrasts include zero.
- Two generation settings differ between the methods (sampler PLMS vs Euler, control duration 15/50 vs 25/50 steps). Part of the gap could come from either; it cannot be bounded without re-running both methods under one sampler and one schedule.
- Train/test isolation was verified at the source and layout level only; COCO image IDs are not available for the benchmark layouts.
- The HSV colour table misjudges yellow/brown/black boundaries, which penalises MIGC more than GLIGEN.
- The fine-tuned models were only evaluated on the validation set; the test-set evaluation of the chosen configurations (G1 and G2 5e-5) is still pending. The grounding-ratio experiment is only quoted qualitatively from the group log.
- The committed test images were generated on a local RTX 4070. The same experiment was also run on an A40 cluster and the numbers recorded in the group notes are very close, but the cluster files are not in the repository, so only the verifiable run is reported.

5.3 Future work

The evidence points to instance isolation as the key ingredient, but this has not been isolated experimentally. The most useful next step is therefore the missing ablation: (a) restrict each GLIGEN grounding token's attention to its own box and see whether that alone removes leakage; (b) fine-tune with colour-annotated phrases to separate "cannot bind" from "never learned"; (c) try an overlap-aware aggregation for MIGC (e.g. soft ownership or depth ordering) for the one condition where it degrades; (d) run both methods under one sampler (both code bases support DDIM) and test on a second benchmark such as HRS or T2I-CompBench.

6. Conclusion

On COCO-MIG, with every shared setting fixed, GLIGEN and MIGC draw the requested objects equally often, but MIGC binds the right attribute to the right object about twice as reliably (0.665 vs 0.321), and this ranking survives changes to the evaluator threshold. The most plausible explanation is architectural — shared grounding attention versus isolated instance branches. It is consistent with the opposite difficulty profiles observed here (GLIGEN fails with more objects, MIGC with overlapping boxes) and with the fact that cheaply fine-tuning GLIGEN's layout modules changes binding only marginally — but it has not been isolated by an ablation. In practice, an instance-isolating controller is preferable when per-object attributes must be correct; when boxes overlap heavily, neither method is yet reliable.

References

[1] Y. Li et al. GLIGEN: Open-Set Grounded Text-to-Image Generation. CVPR 2023.

[2] D. Zhou et al. MIGC: Multi-Instance Generation Controller for Text-to-Image Synthesis. CVPR 2024.

[3] R. Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. CVPR 2022.

[4] S. Liu et al. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. ECCV 2024.

[5] A. Kirillov et al. Segment Anything. ICCV 2023.

[6] J. Wang et al. Exploring CLIP for Assessing the Look and Feel of Images. AAAI 2023.